

Desenvolvimento para Dispositivos Móveis

Future e Navegação

Universidade Federal do Rio Grande do Norte

Future

- Operações assíncronas permitem que o programa continue funcionando **enquanto aguarda a conclusão de outra operação**.
 - Recuperar dados por meio de uma rede.
 - Gravar em um banco de dados.
 - Ler dados de um arquivo.
- Essas computações assíncronas geralmente fornecem seu resultado como um "Future" ou, se o resultado tiver várias partes, como um "Stream".
- Essas computações introduzem a **assincronia em um programa**. Para acomodar essa assincronia inicial, outras funções simples do Dart também **precisam se tornar assíncronas**.
- Para interagir com esses resultados assíncronos, são usadas as palavras-chave "async" e "await". A maioria das funções assíncronas são apenas funções Dart assíncronas que dependem, possivelmente profundamente, de uma computação inerentemente assíncrona.

Future

- Uma "future" (com "f" minúsculo) é uma instância da classe "Future" (com "F" maiúsculo).
- Uma "future" representa o resultado de uma operação assíncrona e pode ter dois estados: não concluída (uncompleted) ou concluída (completed).

Future

- Quando uma "future" está no estado "não concluída", isso significa que a **operação assíncrona** à qual a "future" está vinculada **ainda não foi concluída**.
- Quando a operação é concluída, a "future" entra no estado "concluída" e pode conter um valor ou um erro, dependendo do resultado da operação.
- **As "futures" são uma parte fundamental da programação assíncrona em Dart** e são usadas para tratar resultados de operações que podem levar algum tempo para serem concluídas.

Future

```
Future<void> fetchUserOrder() {  
  // Imagine that this function is fetching user info but encounters a bug  
  return Future.delayed(const Duration(seconds: 2),  
    () => throw Exception('Logout failed: user ID is invalid'));  
}  
  
void main() {  
  fetchUserOrder();  
  print('Fetching user order...');  
}
```

async e await

- As palavras-chave "async" e "await" **fornece uma maneira declarativa** de definir funções assíncronas e usar seus resultados.
- Para definir uma função assíncrona, adiciona-se "async" antes do corpo da função.
- A palavra-chave "await" **funciona apenas em funções assíncronas**.

async e await

```
Future<String> createOrderMessage() async {  
    var order = await fetchUserOrder();  
    return 'Your order is: $order';  
}
```

```
Future<String> fetchUserOrder() =>  
    // Imagine that this function is  
    // more complex and slow.  
    Future.delayed(  
        const Duration(seconds: 2),  
        () => 'Large Latte',  
    );
```

```
Future<void> main() async {  
    print('Fetching user order...');  
    print(await createOrderMessage());  
}
```

Navigator e Router

- Flutter oferece um sistema completo para navegar entre telas e lidar com links profundos.
- Aplicações pequenas, sem vinculação complexa, podem usar o **Navigator**.
- Aplicativos com requisitos específicos de vinculação devem também utilizar o **Router** para lidar corretamente com links.

Navigator

- O widget **Navigator** exibe as telas como uma pilha, utilizando as animações de transição adequadas para a plataforma de destino.
- Para navegar para uma nova tela, acesse o Navigator através do BuildContext da rota e chame métodos como **push()** ou **pop()**

```
onPressed: () {  
  Navigator.of(context).push(  
    MaterialPageRoute(  
      builder: (context) => const Text("Tela"),  
    ),  
  );  
},
```

```
onPressed: () => Navigator.pop(context, 50),
```



Resultado enviado de volta

Navigator

- Como o Navigator mantém uma pilha de objetos **Route** (representando o histórico), o método `push()` também recebe um objeto `Route`.
- O objeto **MaterialPageRoute** é uma subclasse de `Route` que especifica as animações de transição para o Material Design.

```
onPressed: () {  
  Navigator.of(context).push(  
    MaterialPageRoute(  
      builder: (context) => const Text("Tela"),  
    ),  
  );  
},
```

```
onPressed: () => Navigator.pop(context, 50),
```



Resultado enviado de volta

Router

- Aplicações flutter podem navegar utilizando `MaterialApp.routes`.
- Essa navegação é chamada de *named routes*.

Router – Exemplo 1

```
MaterialApp(  
  title: 'Named Routes Demo',  
  initialRoute: '/',  
  routes: {  
    '/': (context) => const FirstScreen(),  
    '/second': (context) => const SecondScreen(),  
  },  
)
```

```
onPressed: () {  
  Navigator.pushNamed(context, '/second');  
}
```

```
onPressed: () {  
  Navigator.pop(context);  
}
```

Router – Exemplo 2

```
MaterialApp(  
  title: 'Named Routes Demo',  
  initialRoute: '/',  
  routes: {  
    '/': (context) => const FirstScreen(),  
    '/second': (context) => const SecondScreen(varName: "Exemplo"),  
  },  

```

```
onPressed: () {  
  Navigator.pushNamed(context, '/second');  
}
```

```
class SecondScreen extends StatelessWidget {  
  const SecondScreen({super.key, required this.varName});  
  
  final String varName;  
  
  @override  
  Widget build(BuildContext context) {  
  }  
}
```

Router – Exemplo 3

```
MaterialApp(  
  title: 'Named Routes Demo',  
  initialRoute: '/',  
  routes: {  
    '/': (context) => const FirstScreen(),  
    '/second': (context) => const SecondScreen(),  
  },  
)
```

```
onPressed: () {  
  Navigator.pushNamed(  
    context,  
    "/second",  
    arguments: "Exemplo",  
  );  
},
```

```
class SecondScreen extends StatelessWidget {  
  const SecondScreen({super.key});  
  @override  
  Widget build(BuildContext context) {  
    final args = ModalRoute.of(context)!.settings.arguments as String;  
    return (..) ;  
  }  
}
```

Navegação e Retorno de Valor

```
class FirstScreen extends StatelessWidget {  
  const FirstScreen({super.key});  
  
  Future<void> _navigateToSecondScreen(BuildContext context) async {  
    final result = await Navigator.push(  
      context,  
      MaterialPageRoute(builder: (context) => const SecondScreen()),  
    );  
    if (!context.mounted) return;  
    ScaffoldMessenger.of(context)  
      ..removeCurrentSnackBar()  
      ..showSnackBar(SnackBar(content: Text('$result')));  
  }  
}
```

```
onPressed: () => _navigateToSecondScreen(context),
```

```
onPressed: () {  
  Navigator.pop(context, "Hello!");  
},
```

Hero Widget

- Hero é um widget que marca seu filho como **candidato para animações**.
- Quando há uma navegação conteúdo de toda a tela é substituído. Uma rota antiga desaparece e uma nova rota aparece. Se houver um recurso visual comum em ambas as rotas, pode ser útil para orientar o usuário que o recurso se mova fisicamente de uma página para outra durante a transição das rotas.
- Essa animação é chamada de animação Hero.
- Os widgets Hero "voam" na sobreposição do Navigator durante a transição e, enquanto estão em movimento, não são exibidos em seus locais originais nas rotas antiga e nova, por padrão.
- Para rotular um widget como tal **recurso, envolva-o em um widget Hero**. Quando a navegação ocorre, **os widgets Hero em cada rota são identificados pelo HeroController**. Para cada par de widgets Hero que têm a mesma tag, uma animação de herói é desencadeada.

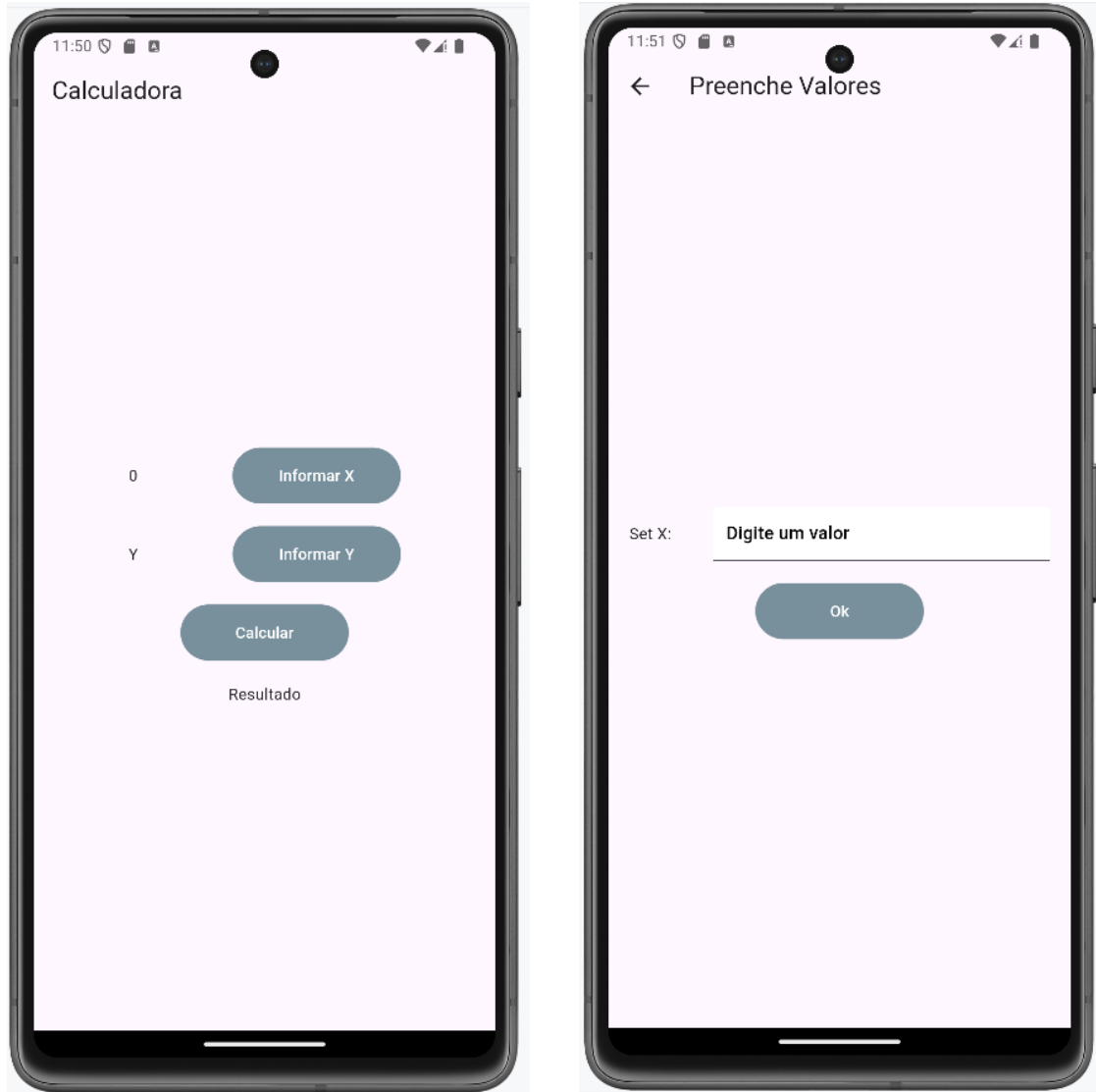
Hero Widget

```
class MainScreen extends StatelessWidget {  
  (...)  
  body: GestureDetector(  
    onTap: () {  
      Navigator.push(context, MaterialPageRoute(builder: (context) {  
        return const DetailScreen();  
      }));  
    },  
    child: Hero(  
      tag: 'imageHero',  
      child: Container(  
        width: 150,  
        height: 150,  
        decoration: const BoxDecoration(  
          shape: BoxShape.circle,  
          image: DecorationImage(  
            fit: BoxFit.fill,  
            image: NetworkImage(  
              'https://inclusaodigital.imd.ufrn.br/img/logos/imd.png',  
            ),  
          ),  
        ),  
      ),  
    ),  
  ),  
}
```

Hero Widget

```
class DetailScreen extends StatelessWidget {  
  child: Column(  
    mainAxisAlignment: MainAxisAlignment.center,  
    children: [  
      const Text(  
        "Instituto Metr pole Digital",  
        style: TextStyle(  
          fontSize: 32,  
          fontWeight: FontWeight.bold,  
        ),  
      ),  
      Hero(  
        tag: 'imageHero',  
        child: Image.network(  
          'https://inclusaodigital.imd.ufrn.br/img/logos/imd.png',  
        )),  
      const Padding(  
        padding: EdgeInsets.all(8.0),  
        child: Text(  
          "Lorem ipsum dolor sit amet, consectetur adipiscing elit. "  
          textAlign: TextAlign.justify,  
        ),  
      ),  
    ],  
  ),  
)
```

Atividade Calculadora – Parte 2



- Crie um aplicativo com duas telas.
- Navegue da tela 1 para tela 2 para informar valores a serem usados no cálculo.
- Na tela 2 retorne para tela 1 após obter um valor.
- O botão calcular escreve o resultado.

Referências

- <https://dart.dev/>
- <https://dart-tutorial.com/>
- <https://docs.flutter.dev/ui>
- <https://docs.flutter.dev/ui/widgets>
- <https://docs.flutter.dev/ui/layout>
- <https://docs.flutter.dev/resources/architectural-overview>